Exercices Séance 9 : Programmation de UNION FIND

Pour ce TD, n'hésitez pas à retourner voir le sujet de la semaine dernière présentant 3 façons d'implémenter l'algorithme Union-find :
https://docs.google.com/document/d/1bdLmuWb7EsREbXXE360VrbqruDJxqA-K/edit?usp=sharing&ouid=118441071948612917589&rtpof=true&sd=true

Exercice 1 : Programmer 3 versions d'union-find

A l'aide du fichier python ***td_union_find.py***, il vous est demandé de programmer les trois versions de l'algorithme union-find étudiées au cours de la séance précédente :

- La version naïve avec tableau
- La première version avec représentation d'arbres
- La version d'arbres incluant union-by-rank

Pour chacune des versions, vous devrez compléter la fonction « Find » et « Union » correspondante (l9 et 12 pour la version « naïve », l51 et 54 pour la version avec arbres « simples », et l97 et 100 pour la version d'arbres avec « union-by-rank »).
Une fonction « Test_Union_Find » vous permettra de tester chacun de vos algorithmes (l19, l61, et l107 respectivement), en partant de N ensembles d'entiers totalement disjoints et en répétant N opérations UNION(x,y) avec x et y choisis aléatoirement.

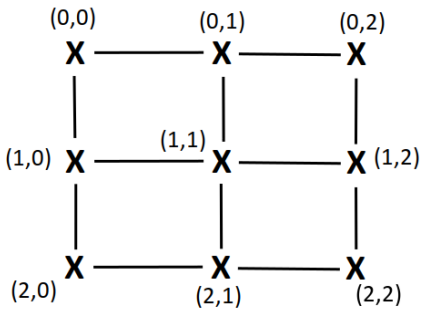
En exécutant le fichier complet, affichez les structures générées par l'exécution de Test_Union_Find avec chacune des 3 versions pour N=5, puis mesurez et comparez les temps d'exécution des 3 versions pour N=1000, 10000 et 20000.

Exercice 2 : Application de Union-Find :

On cherche à générer de manière aléatoire des labyrinthes avec la méthode de « Fusion d'arbres » :

On considère un graphe grille G(n)=(V,E) avec $V = [0, n - 1] \times [0, n - 1]$ et
$((x, y), (x', y')) \in E \ ssi \left(|x - x'| = 1 \ et \ y = y' \right) ou (x = x' \ et \ |y - y'| = 1)$.

Exemple pour n=3 :



1) Dans le fichier ***td_union_find_labyrinthe.py***, complétez la fonction « generer_grille_complete » qui renvoie les ensembles V et E pour une grille complète de taille N donnée.

On va construire le labyrinthe H=(V',E') comme un sous-graphe de G(n) en partant de V'=V et E'=∅. Chaque sommet de H est considéré comme une cellule entourée de murs. Autrement dit, pour n=3, le labyrinthe défini par H ressemble initialement à ceci :

(0,0)	(0,1)	(0,2)
(1,0)	(1,1)	(1,2)
(2,0)	(2,1)	(2,2)

Si vous avez bien répondu à la question 1, alors en exécutant le fichier et en appuyant sur le bouton « Grille initiale » de la fenêtre Tkinter qui s'ouvre devant vous, vous devriez pouvoir afficher le labyrinthe initial pour n=10 (mettez une capture d'écran dans votre rapport pour plusieurs valeurs de n).

Par la suite, nous allons effectuer une boucle où à chaque itération, on sélectionne un mur (c'est-à-dire une arête non visitée e de E), et on teste si sa destruction entraîne la fusion d'arbres différents. Si tel est le cas, on ajoute e dans E'.
Exemple en sélectionnant l'arête e={(0,1),(1,1)} :

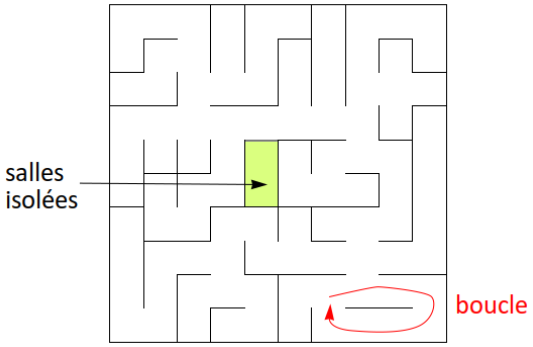
(0,0)	(0,1)	(0,2)
(1,0)	(1,1)	(1,2)
(2,0)	(2,1)	(2,2)

On considère que les arêtes sont représentées dans un tableau T de taille $m = 2n(n-1)$
L'algorithme pour générer le labyrinthe est le suivant :

```
Génère-Labyrinthe (Entrée : taille n du labyrinthe)

   1. Construire G(n) en encodant les sommets dans un tableau de
      paires d'entiers et les arêtes par un tableau T de paires de
      sommets. Initialiser H avec pour arêtes E' un tableau vide.
   2. On applique une permutation aléatoire à T (cf. algorithme de
      Fisher-Yates ci-dessous)
   3. Pour i allant de 1 à m
            e=(u,v) est l'arête stockée dans T[i]
            Si FIND(u) ≠ FIND(v) alors
                      Ajouter e à E'
                      UNION(u,v)
   4. Retourner H
```

2) Expliquez en quoi l'algorithme ci-dessus permettra de générer un labyrinthe sans « boucle » ni « salles isolées » (contrairement à l'exemple ci-dessous). A quelles propriétés du graphe H cela correspond-il ? En supposant que l'entrée du labyrinthe défini par H est située en haut à gauche (salle (0,0)) et la sortie en bas à droite (salle (n-1,n-1)), combien de chemins simples existe-t-il entre l'entrée et la sortie ?



3) Compléter la fonction generer_labyrinthe(N) dans votre fichier Python pour permettre d'exécuter l'algorithme ci-dessus. Notez que les fonctions définies dans l'exercice précédent (issues du fichier ***td_union_find.py***) sont importées au début de celui-ci.
Générez des labyrinthes de taille 10*10 et 100*100 et affichez-les.

Algorithme de Fisher-Yates

Le mélange de Fisher-Yates, également appelé *mélange de Knuth*, est un algorithme en place permettant d'appliquer une permutation aléatoire à n éléments en temps linéaire, les $n!$ permutations possibles étant équiprobables en sortie.

Le principe de cet algorithme est de partir d'un tableau d'éléments T et d'échanger aléatoirement des entrées :

```
pour i de n - 1 à 1 :
      j ← nombre aléatoire entier 0 ≤ j ≤ i
      échanger P[j] et P[i]
```